

Module-1 Basic Structure of Computers and Machine Instructions**Computer:**

A computer can be defined as a fast electronic calculating machine that accepts the (data) digitized input information, process it as per the list of internally stored instructions and produces the resulting information. List of instructions are called programs & internal storage is called computer memory.

Classification of Computer:

Many types of computers exist that differ widely in size, cost, computational power and usage.

➤ Personal Computer or Microcomputer:

- It is most common computer.
- It is a single-user computer based on a microprocessor/microchip inside.
- It is designed for one person; therefore, it is called a personal computer.
- These Computers are small in size, small in memory, and less expensive.
- Many input and output devices can be attached to it.
- It is generally used in homes, offices, schools and entertainment like playing music or games.
- It is further classified into the following categories.
- **Desktop Computer:** a computer that can easily be accommodated on the table.
- **Laptop Computer:** a computer that can easily be placed on a lap.
- Examples: IBM, APPLE, PENTIUM, Core series, etc.

➤ Workstation:

- The Workstation has more computational power as compared to microprocessors.
- Workstations are often used in engineering applications, especially for interactive design work.

➤ Minicomputer:

- A Minicomputer is a multi-user / multiprocessing computer capable to handle 10 to 200 users simultaneously.
- These types of computers have a medium size, less memory, and are more powerful and expensive than microcomputers.



- Minicomputer contains one or more processors and supports multiprocessing and tasking.
- They are more powerful than personal computers and workstations. Multiple Input & Output devices can be attached to them and used in Networking.
- Example: IBM SYSTEM36.

➤ **Mainframe Computer:**

- [Mainframe Computer](#) is an expensive and multi-user / multiprocessing computer capable of supporting more than hundreds or thousands of users at the same time.
- These computers have large sizes and memory and are more powerful than Minicomputers. These computers have high-speed data processing power and can process millions of arithmetic expressions per second.
- A mainframe computer is expensive and is used by large organizations or industries for significant jobs and data processing. For the computerization of huge businesses, Universities, Banks, Scientific Laboratories, National & International Markets, etc. it is usually filled with a large room because many peripheral, input, and output devices can be attached to it.
- Examples: IBM 709, IBM 4381, IBMz Series,



➤ **Supercomputer:**

- A [supercomputer](#) is the most powerful multi-user and fastest computer in the world.
- It calculates a trillion million instructions per second up to 14 accurate decimal places.
- Multi CPUs are working together on this computer.
- It is very expensive with normal costs of several million dollars. It is made for the general purpose to solve the problems of Scientists & Engineers and is referred to as “FORTRAN ENGINES” which means FORMula TRANslation because FORTRAN language is commonly used by scientists and engineers.
- A supercomputer is considered to be the main tool for the reproduction of nuclear weapons denotation, aerodynamic flows, and global weather patterns.
- Examples: JAGUAR, IBM's ROADRUNNER.

Functional unit:

A computer consists of five functionally independent main parts input, memory, arithmetic logic unit (ALU), and output and control unit.

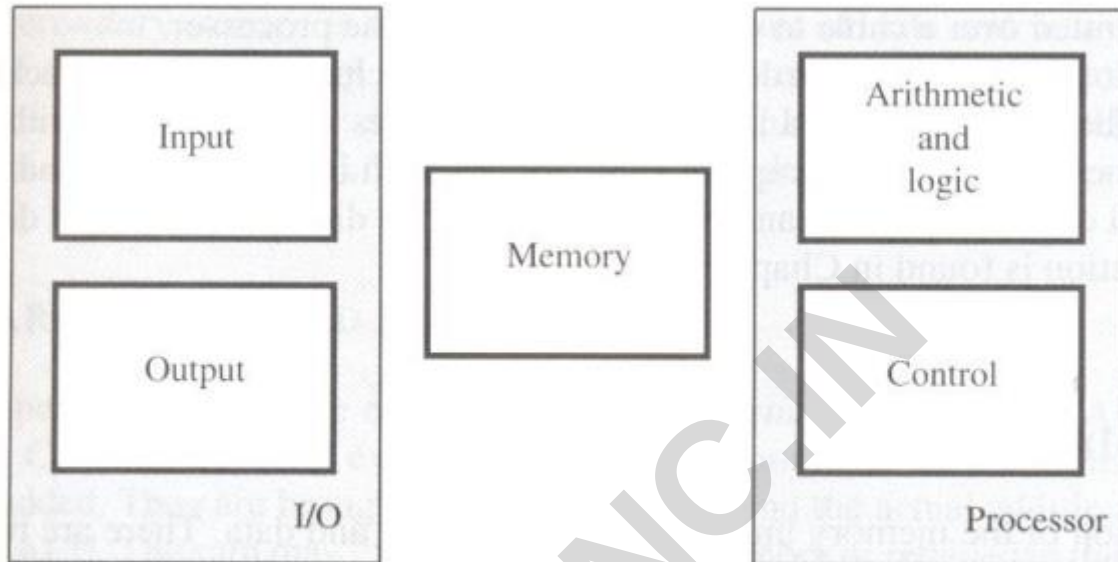


Figure Basic functional units of a computer.



Input device accepts the coded information as source program i.e. high level language. This is either stored in the memory or immediately used by the processor to perform the desired operations. The program stored in the memory determines the processing steps. Basically the computer converts one source program to an object program, i.e. into machine language.

Finally the results are sent to the outside world through output device. All of these actions are coordinated by the control unit.

Input unit: -

The source program/high level languages program/coded information/simply data is fed to a computer through input devices keyboard is a most common type. Whenever a key is pressed, one corresponding word or number is translated into its equivalent binary code over a cable & fed either to memory or processor. Joysticks, trackballs, mouse, scanners etc are other input devices.

Memory unit: -

Its function is to store programs and data. There are two classes of storage, they are:

**1. Primary memory 2. Secondary memory**

1. Primary memory: - Is the one exclusively associated with the processor and operates at the electronics speeds programs must be stored in this memory while they are being executed. The memory contains a large number of semiconductors storage cells, each capable of storing one bit of information. These cells are rarely read or written as individual cells but instead are processed in groups of fixed size called words.

To provide easy access to a word in memory, a distinct address is associated with each word location. **Addresses are** numbers that identify memory location. Number of bits in each word is called word length of the computer. Programs must reside in the memory during execution. Instructions and data can be written into the memory or read out under the control of processor.

Memory in which any location can be reached in a short and fixed amount of time after specifying its address is called random-access memory (RAM).

The time required to access one word is called memory access time. Memory which is only readable by the user and contents of which can't be altered is called read only memory (ROM) it contains operating system.

Caches are the small fast RAM units, which are coupled with the processor and are often contained on the same IC chip to achieve high performance. Although primary storage is essential it tends to be expensive.

2. Secondary Memory: - Is used where large amounts of data & programs have to be stored, particularly information that is accessed infrequently.

Examples: Magnetic disks & tapes, optical disks (ie CD-ROM's), floppies etc.,

Arithmetic logic unit (ALU):-

Most of the computer operators are executed in ALU of the processor like addition, subtraction, division, multiplication, etc. the operands are brought into the ALU from memory and stored in high speed storage elements called register. Then according to the

instructions the operation is performed in the required sequence.

The control and the ALU are many times faster than other devices connected to a computer system. This enables a single processor to control a number of external devices such as key boards, displays, magnetic and optical disks, sensors and other mechanical controllers.

Output unit:-

These actually are the counterparts of input unit. Its basic function is to send the processed results to the outside world.

Examples: Printer, speakers, monitor etc.,

Control unit:-

It effectively is the nerve center that sends signals to other units and senses their states. The actual timing signals that govern the transfer of data between input unit, processor, memory and output unit are generated by the control unit.

Basic operational concepts:

To perform a given task an appropriate program consisting of a list of instructions is stored in the memory. Individual instructions are brought from the memory into the processor, which executes the specified operations. Data to be stored are also stored in the memory.

Examples: - Add LOCA, R0

This instruction adds the operand at memory location LOCA, to operand in register R0 & places the sum into register. The original contents of location LOCA are preserved, whereas those of R0 are overwritten. This instruction requires the performance of several steps,

1. First the instruction is fetched from the memory into the processor.
2. The operand at LOCA is fetched and added to the contents of R0
3. Finally the resulting sum is stored in the register R0

The preceding Add instruction combines a memory access operation with an ALU Operations. In some other type of computers, these two types of operations are performed

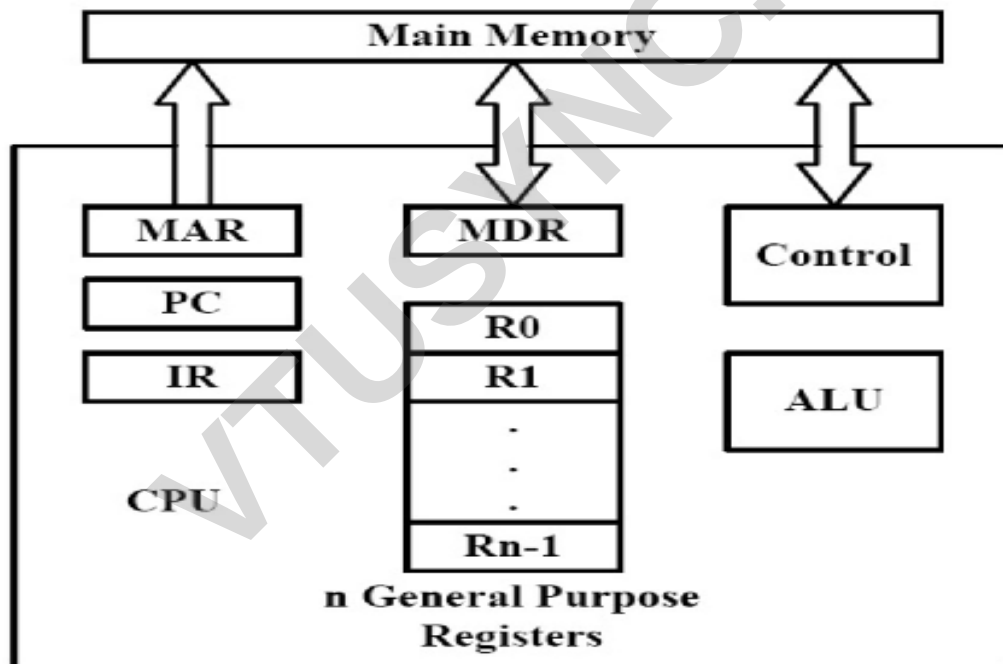
by separate instructions for performance reasons.

Load LOCA, R1

Add R1, R0

The first of these instructions transfers the contents of memory location LOCA into processor register R1, and the second instruction adds the contents of registers R1 and R0 and places the sum into R0.

Transfers between the memory and the processor are started by sending the address of the memory location to be accessed to the memory unit and issuing the appropriate control signals. The data are then transferred to or from the memory.



The fig shows how memory & the processor can be connected. In addition to the ALU & the control circuitry, the processor contains a number of registers used for several different purposes.

The instruction register (IR):- Holds the instruction that is currently being executed. Its output is available for the control circuits which generates the timing signals that control the various processing elements in one execution of instruction.

The program counter PC:-

This is another specialized register that keeps track of execution of a program. It contains the memory address of the next instruction to be fetched and executed. Besides IR and PC, there are n -general purpose registers R_0 through R_{n-1} .

The other two registers which facilitate communication with memory are:

- 1. MAR – (Memory Address Register):-** It holds the address of the location to be accessed.
- 2. MDR – (Memory Data Register):-** It contains the data to be written into or read out of the address location.

Operating steps are

1. Programs reside in the memory & usually get these through the Input unit.
2. Execution of the program starts when the PC is set to point at the first instruction of the program.
3. Contents of PC are transferred to MAR and a Read Control Signal is sent to the memory.
4. After the time required to access the memory elapses, the address word is read out of the memory and loaded into the MDR.
5. Now contents of MDR are transferred to the IR & now the instruction is ready to be decoded and executed.
6. If the instruction involves an operation by the ALU, it is necessary to obtain the required operands.
7. An operand in the memory is fetched by sending its address to MAR & Initiating a read cycle.
8. When the operand has been read from the memory to the MDR, it is transferred from MDR to the ALU.
9. After one or two such repeated cycles, the ALU can perform the desired operation.
10. If the result of this operation is to be stored in the memory, the result is sent to MDR.
11. Address of location where the result is stored is sent to MAR & a write cycle is initiated.
12. The contents of PC are incremented so that PC points to the next instruction that is to be executed.

Normal execution of a program may be preempted (temporarily interrupted) if some devices require urgent servicing, to do this one device raises an Interrupt signal.

An interrupt is a request signal from an I/O device for service by the processor. The processor provides the requested service by executing an appropriate interrupt service routine.

The Diversion may change the internal stage of the processor its state must be saved in the memory location before interruption. When the interrupt-routine service is completed the state of the processor is restored so that the interrupted program may continue.

Bus Structures:

To achieve a reasonable speed of operation, a computer must be organized so that all its units can handle one full word of data at a given time. A group of lines that serve as a connecting port for several devices is called a bus. Bus is defined as **set of parallel wires** used for **data communication**. Each wire carries **1 bit** of data.

There are 3 of buses, namely

1. Address bus
2. Data bus and
3. Control bus.

1. Address bus :

- ❖ It is **unidirectional**.
- ❖ The **processor** (CPU) sends the **address** of an **I/O device** or **Memory device** by means of this bus.

2. Data bus

- ❖ It is a **bidirectional** bus.
- ❖ The CPU sends data from **Memory to CPU** and vice versa as well as **from I/O to CPU** and vice versa by means of this bus.

3. Control bus:

- ❖ This bus carries control signals for **Memory** and **I/O** devices.
- ❖ It generates control signals for Memory namely **MEMRD** and **MEMWR** and control signals for I/O devices namely **IORD** and **IOWR**.
- ❖ It also generates special control signal to differentiate between Memory and I/O device.



- ❖ This control signal is called as M/I/O.

M/I/O = **1** for **Memory** operations

M/I/O = **0** for **I/O** operations



‘The simplest way to interconnect functional units is to use a single bus, as shown in Figure. All units are connected to this bus. Because the bus can be used for only one transfer at a time, only two units can actively use the bus at any given time. Bus control lines are used to arbitrate multiple requests for use of one bus.

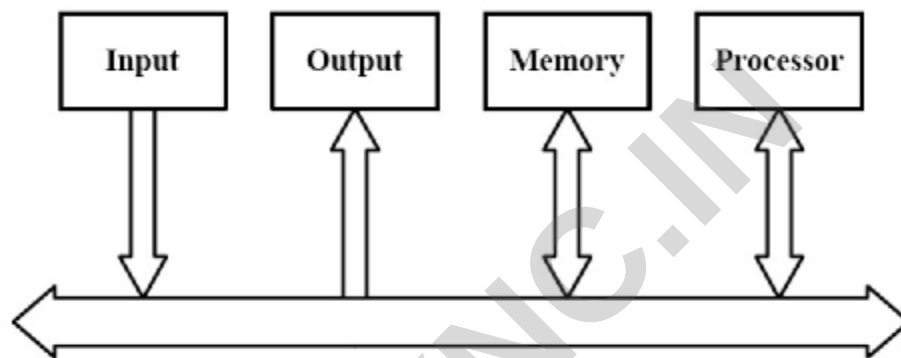


Fig: Single bus structure

Single bus structure is

- Low cost
- Very flexible for attaching peripheral devices
- Poor performance due to single bus

Multiple bus structure certainly increases the performance but also increases the cost significantly.

All the interconnected devices are not of same speed & time leads to a bit of a problem. This is solved by using cache registers (ie buffer registers). These buffers are electronic registers of small capacity when compared to the main memory but of comparable speed.

The instructions from the processor at once are loaded into these buffers and then the complete transfer of data at a fast rate will take place.

Software:

System software is a collection of programs that are executed as needed to perform functions such as,

- Receiving and interpreting user commands
- Entering and editing application programs and storing them as files in secondary storage devices
- Managing the storage and retrieval of files in secondary storage devices
- Running standard application programs such as word processors, spreadsheets, or games, with data supplied by the user
- Controlling I/O units to receive input information and produce output results
- Translating programs from source form prepared by the user into object form consisting of machine instructions
- Linking and running user-written application programs with existing standard library routines, such as numerical computation packages

System software is thus responsible for the coordination of all activities in a computing system. Application programs are usually written in a high-level programming language, such as C, C++, Java, or FORTRAN, in which the programmer specifies mathematical or text-processing operations. A system software program called a **compiler** translates the high-level language program into a suitable machine language program.

Another important system program is **Text editor**; it is used for entering and editing application programs. The user of this program interactively executes commands that allow statements of a source program entered at a keyboard to be accumulated in a file.

A **file** is simply a sequence of alphanumeric characters or binary data that is stored in memory or in secondary storage. A file can be referred to by a name chosen by the user.

Operating system (OS) is a large program, or actually a collection of routines, that is used to control the sharing of and interaction among various computer units as they execute application programs. The OS routines perform the tasks required to assign computer resources to individual application programs. These tasks include assigning memory and magnetic disk space to program and data files, moving data between memory and disk units, and handling I/O operations.

Software – Operating System

Operating System (OS) is a collection of routines used to control the sharing of and interaction among various computer units as they execute application programs

- Assign computer resources to individual application programs
- Assign memory and magnetic disk space to program
- Move data between memory and disk units
- Handle I/O operations

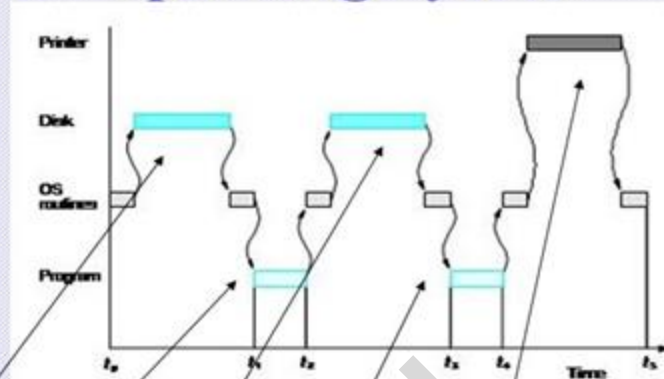


Figure User program and OS routine sharing of the processor.

OS loads application program from disk

OS loads data file from disk

Transfer data file from disk and prints the results

Waits for transfer to occur, then passes execution control to application program

Waits for transfer to occur, then passes execution control to application program

During the time period t_0 to t_1 , OS routine initiates loading the application program from disk to memory, waits until the transfer is completed, and then passes execution control to the application program. A similar pattern of activity occurs during period t_2 to t_3 and period t_4 to t_5 , when the operating system transfers the data file from the disk and prints the results. At t_5 , the operating system may load and execute another application program.

Now, let us point out a way that computer resources can be used more efficiently if several application programs are to be processed. Notice that the disk and the processor are idle during most of the time period t_4 to t_5 . The operating system can load the next program to be executed into the memory from the disk while the printer is operating. Similarly, during t_0 to t_1 , the operating system can arrange to print the previous program's results while the current program is being loaded from the disk.

Thus, the operating system manages the concurrent execution of several application programs to make the best possible use of computer resources. This pattern of concurrent execution is called multiprogramming or multitasking.

Performance:

The most important measure of the performance of a computer is how quickly it can execute programs. The speed with which a computer executes program is affected by the design of its hardware.

The total time required to execute the program is elapsed time is a measure of the performance of the entire computer system. It is affected by the speed of the processor, the disk and the printer. The time needed to execute an instruction is called the processor time.

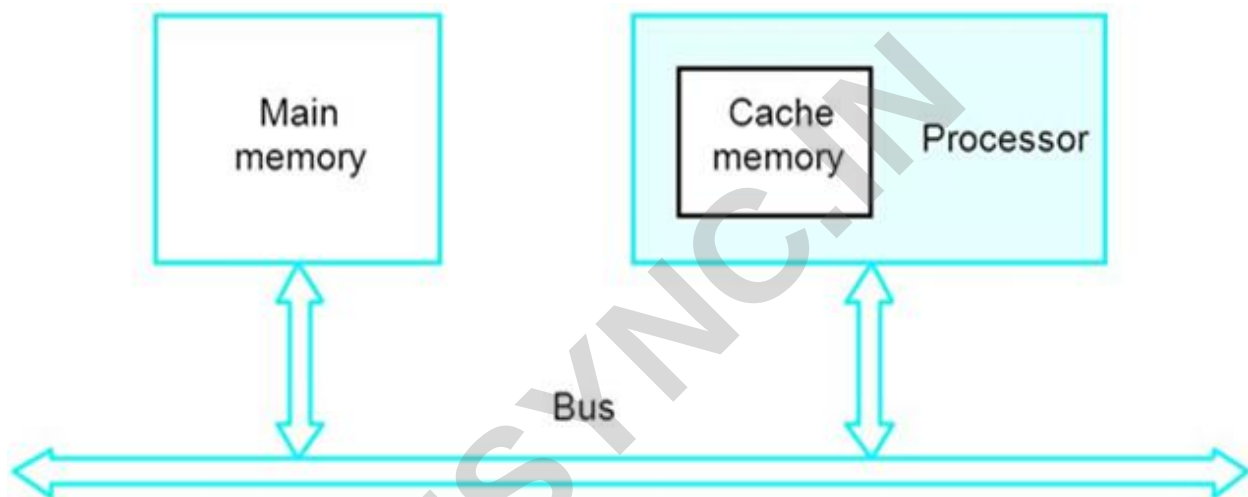


Figure The processor cache.

Let us examine the flow of program instructions and data between the memory and the processor. At the start of execution, all program instructions and the required data are stored in the main memory. As the execution proceeds, instructions are fetched one by one over the bus into the processor, and a copy is placed in the cache later if the same instruction or data item is needed a second time, it is read directly from the cache.

The processor and relatively small cache memory can be fabricated on a single IC chip. The internal speed of performing the basic steps of instruction processing on chip is very high and is considerably faster than the speed at which the instruction and data can be fetched from the main memory. A program will be executed faster if the movement of instructions and data between the main memory and the processor is minimized, which is achieved by using the cache.

For example: Suppose a number of instructions are executed repeatedly over a short period of time as happens in a program loop. If these instructions are available in the cache, they can be fetched quickly during the period of repeated use. The same applies to the data that are used repeatedly.

The Performance of the Computer System is based on the following **factors**.



1. Cache Memory
2. Processor clock
3. Basic Performance Equation
4. Pipelining
5. Compiler

Cache Memory:

- ❖ It is defined as a **fast access memory** located in between CPU and Memory.
- ❖ The processor needs **more time to read** the data and instructions from **main memory** because main memory is away from the processor.
- ❖ Hence it **slowdown** the performance of the system. The processor needs **less time** to read the data and instructions from **Cache Memory** because it is part of the processor.
- ❖ Hence it **improves** the performance of the system.

Processor clock:

Processor circuits are controlled by a timing signal called clock. The clock designer the regular time intervals called clock cycles. To execute a machine instruction the processor divides the action to be performed into a sequence of basic steps that each step can be completed in one clock cycle. The length P of one clock cycle is an important parameter that affects the processor performance.

Processor used in today's personal computer and work station has a clock rates that range from a few hundred million to over a billion cycles per second.

To execute one instruction there are 3 basic steps namely

1. Fetch
2. Decode
3. Execute.

Basic Performance Equation:

Let T = Processor time required to executed a program.

N = Actual number of instruction executions.

S = Average number of basic steps needed to execute one machine instruction.

R = Clock rate in cycles per second.

- The program execution time is given by

$$T = \frac{N * S}{R}$$

- Equ1 is referred to as the basic performance equation.
- To achieve high performance, the computer designer must reduce the value of T, which means reducing N and S, and increasing R.

The value of N is reduced if source program is compiled into fewer machine instructions.

The value of S is reduced if instructions have a smaller number of basic steps to perform.

Pipelining:

- ❖ Instructions are not necessarily executed one after another.
- ❖ The value of S doesn't have to be the number of clock cycles to execute one instruction.
- ❖ Pipelining – overlapping the execution of successive instructions.
- ❖ In the below figure, while decoding ADD instruction fetching of SUB instruction taking place.

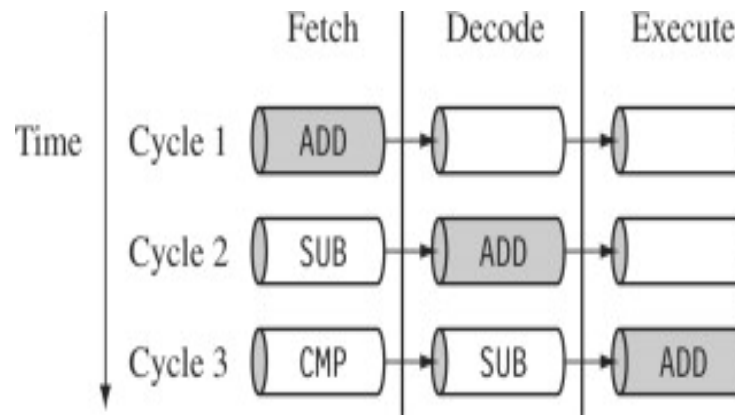


Fig: Pipelining:

Compiler:

- ❖ A compiler translates a high-level language program into a sequence of machine instructions.
- ❖ To reduce N , we need a suitable machine instruction set and a compiler that makes good use of it.
- ❖ Goal – reduce $N \times S$
- ❖ A compiler may not be designed for a specific processor; however, a high-quality compiler is usually designed for, and with, a specific processor

Memory-Locations & Addresses

- Memory consists of many millions of storage cells (flip-flops).
- Each cell can store a bit of information i.e. 0 or 1.
- Each group of n bits is referred to as a word of information, and n is called the word length.
- The word length can vary from 8 to 64 bits.
- A unit of 8 bits is called a byte.
- Accessing the memory to store or retrieve a single item of information (word/byte) requires distinct addresses for each item location. (It is customary to use numbers from 0 through $2^K - 1$ as the addresses of successive-locations in the memory).
- If $2^K = \text{no. of addressable locations}$; then 2^K addresses constitute the address-space of the computer. For example, a 24-bit address generates an address-space of 2^{24} locations (16 MB).

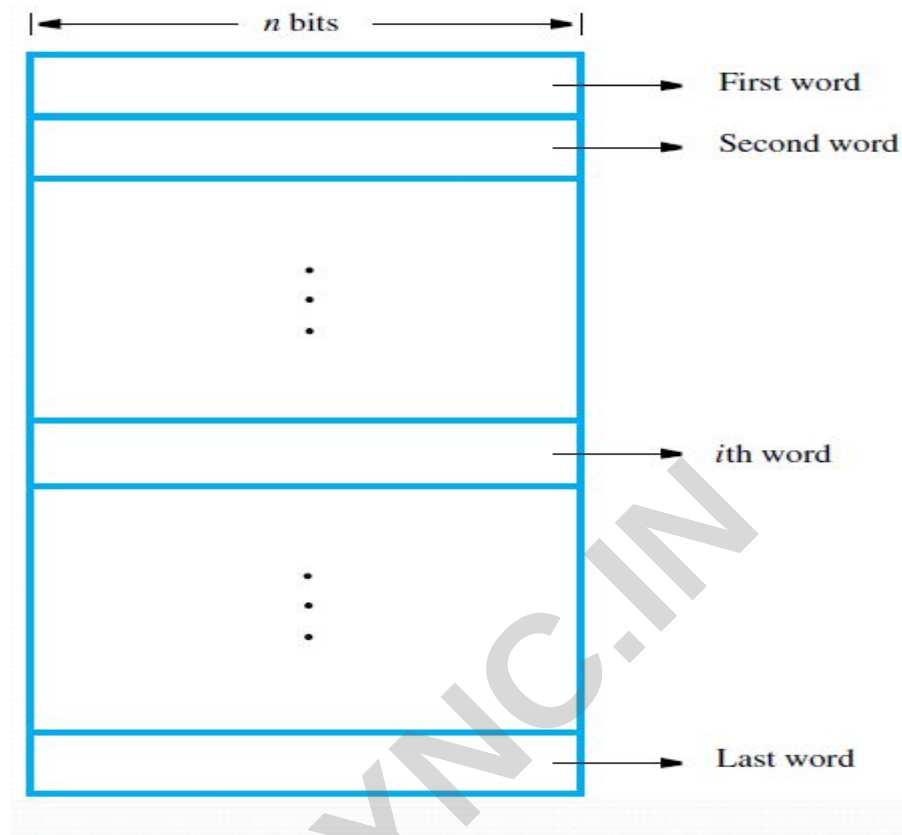
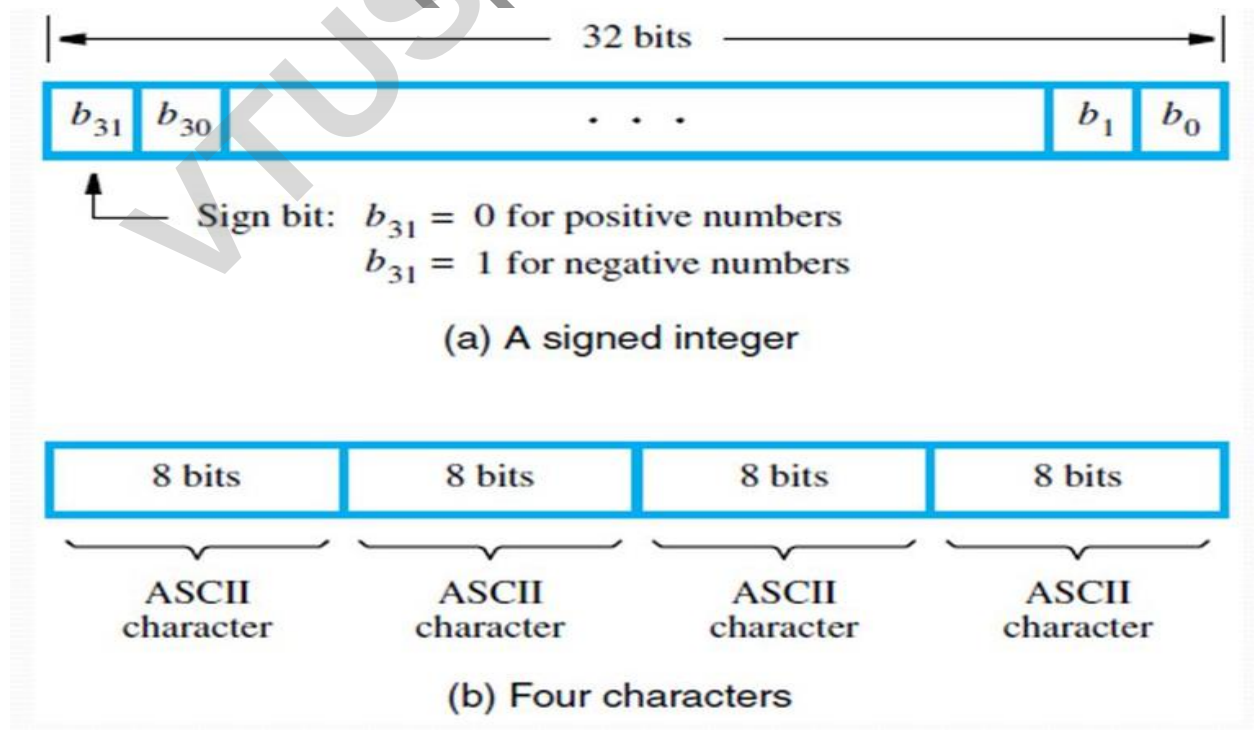


Fig: Memory Locations & Addresses

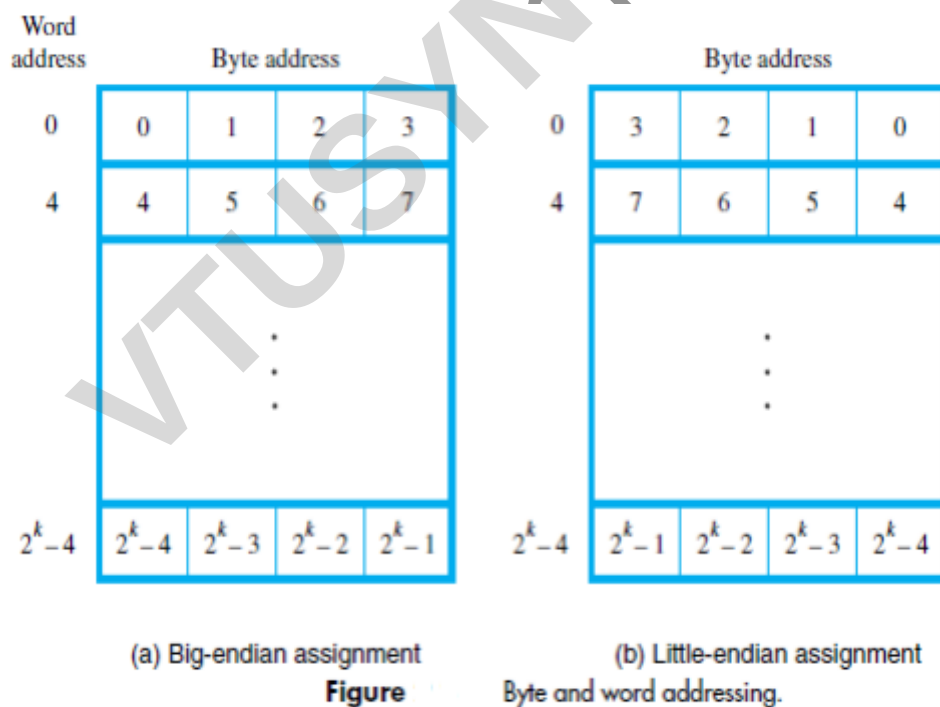


Byte-Addressability:

- In byte-addressable memory, successive addresses refer to successive byte locations in the memory.
- Byte locations have addresses 0, 1, 2,
- If the word-length is 32 bits, successive words are located at addresses 0, 4, 8, . . with each word having 4 bytes.

Big-Endian & Little-Endian Assignments

- There are two ways in which byte-addresses are arranged Figure.
- 1) Big-Endian: Lower byte-addresses are used for the more significant bytes of the word.
 - 2) Little-Endian: Lower byte-addresses are used for the less significant bytes of the word
- In both cases, byte-addresses 0, 4, 8,are taken as the addresses of successive words in the memory.



Consider a 32-bit integer (in hex): 0x12345678 which consists of 4 bytes: 12, 34, 56, and 78.

Hence this integer will occupy 4 bytes in memory.

Assume, we store it at memory address starting 1000.



On little-endian, memory will look like

Address	Value
1001	78
1002	56
1003	34
1004	12

On Big-endian, memory will look like

Address	Value
1001	12
1002	34
1003	56
1004	78

Instructions and Instruction Sequencing

A computer must have instructions capable of performing four types of operations.

- Data transfers between the memory and the processor registers(MOV,PUSH,POP)
- Arithmetic and logic operations on data(ADD,SUB)
- Program sequencing and control(LOOP)
- I/O transfers(IN & OUT)

Register Transfer Notation:-

Transfer of information from one location in the computer to another. Possible locations that may be involved in such transfers are memory locations that may be involved in such transfers are memory locations, processor registers, or registers in the I/O subsystem. Most of the time, we identify a location by a symbolic name standing for its hardware binary address. For *Example*,

names for the addresses of memory locations may be LOC, PLACE, A, VAR2; processor registers names may be R0, R5; and I/O register names may be DATAIN, OUTSTATUS, and so on. The contents of a location are denoted by placing square brackets around the name of the location. Thus, the expression



$$R_1 \leftarrow [LOC]$$

Means that the contents of memory location LOC are transferred into processor register R1.

As another example, consider the operation that adds the contents of registers R1 and R2, and then places their sum into register R3. This action is indicated as

$$R_3 \leftarrow [R_1 + R_2]$$

This type of notation is known as *Register Transfer Notation (RTN)*. Note that “the right-hand side of an RTN expression always denotes a value, and the left-hand side is the name of a location where the value is to be placed, overwriting the old contents of that location”.

Instruction Format:

Opcode	Source operand,	Destination operand
--------	-----------------	---------------------

Where

Opcode stands for operation code, tells what operation to be carried out.

Operands may be register or Memory location.

Instructions are three types:

- One address instruction
- Two address instruction
- Three address instruction

One address instruction:

Instruction Format:

Opcode	Memory operand
--------	----------------

Example: ADD A

Add the contents of memory location A to the contents of the accumulator register and place the sum back into the accumulator.

Example: LOAD A

Example: STORE A

The Load instruction copies the contents of memory location A into the accumulator, and the Store instruction copies the contents of the accumulator into memory location A.

Using only one-address instructions, the operation $C \leftarrow [A] + [B]$ can be performed by executing the sequence of instructions

Load A

Add B

Store C

**Two address instruction:**

Instruction Format

Opcode	Source operand,	Destination operand
--------	-----------------	---------------------

Example: Add A, B

Which performs the operation $B \leftarrow [A] + [B]$. When the sum is calculated, the result is sent to the memory and stored in location B, replacing the original contents of this location. This means that operand B is both a source and a destination.

Example: Move B, C

Move instruction that copies the contents of one memory location into another. Which performs the operations $C \leftarrow [B]$, leaving the contents of location B unchanged.

Three address instruction:

Instruction Format:

Opcode	Source operand,	Source operand,	Destination operand
--------	-----------------	-----------------	---------------------

Example: ADD A,B,C

Operands A and B are called the source operands, C is called the destination operand, and ADD is the operation to be performed on the operands.

Instruction Execution and Straight-Line Sequencing:

The three instructions of the program are in successive word locations, starting at location i. Since each instruction is 4 bytes long, the second and third instructions start at addresses $i + 4$ and $i + 8$.

Let us consider how below program is executed. The processor contains a register called the program counter (PC), which holds the address of the instruction to be executed next.

To begin executing a program, the address of its first instruction (i in our example) must

be placed into the PC. Then, the processor control circuits use the information in the PC to fetch and execute instructions, one at a time, in the order of increasing addresses. This is called straight-line sequencing. During the execution of each instruction, the PC is incremented by 4 to point to the next instruction. Thus, after the Move instruction at location $i + 8$ is executed, the PC contains the value $i + 12$, which is the address of the first instruction of the next program segment.

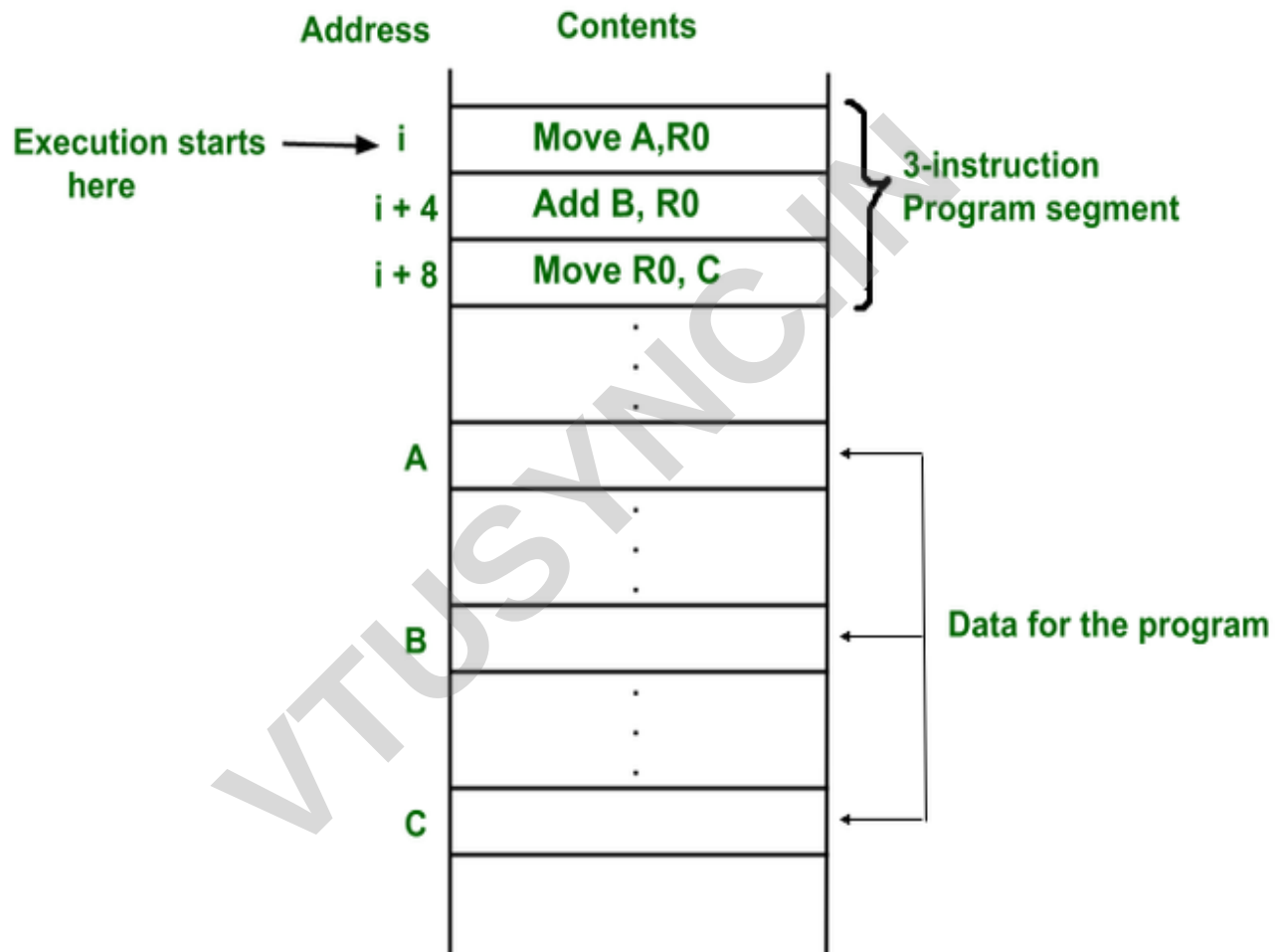


Fig: A program for $C \leftarrow [A] + [B]$.

Executing a given instruction is a two-phase procedure: **instruction fetch & instruction execute**.

In the first phase the instruction is fetched from the memory location whose address is in the PC. This instruction is placed in the instruction register (IR) in the processor.

At the start of the second phase the instruction in IR is examined to determine which operation is to be performed.

Branching:

Straight line sequencing program for adding n numbers.

- The addresses of the memory locations containing the n numbers are represented as NUM1, NUM2, ..., NUMn (i.e. NUM1 address includes first number).
- The first number is stored into processor register R0. And every other number is added to register R0. Finally, when the program ends (i.e. n numbers are added, the result is placed in memory location SUM

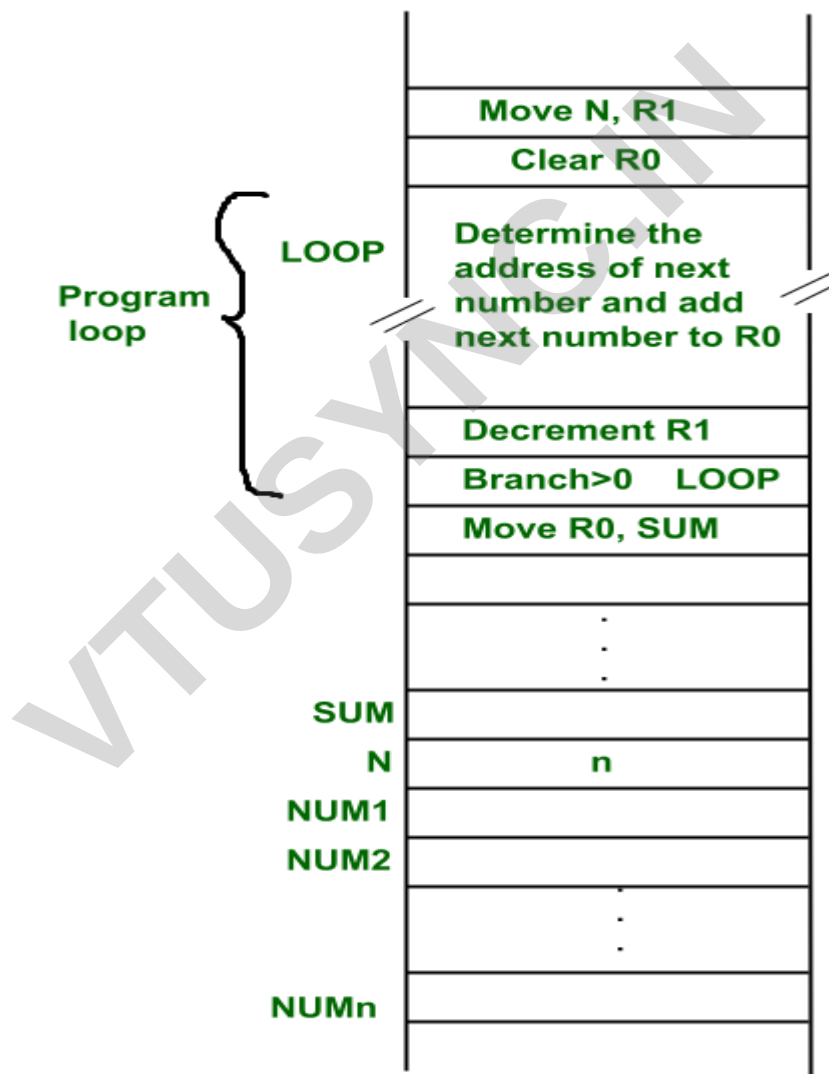


Fig: Using a loop to add n numbers

- The second way is to use a loop to add n number. But here straight line sequencing is not used because every time loop iteration ends, PC has to load the branch address and

program execution starts from that address.

- Here the location N stores the value of n. Processor register R1 is used as a counter to determine the number of times the loop gets executed.
- The contents of the location N are moved into R1 at the start of program execution.
- After that, register R0 is cleared.
- The address LOOP is reloaded again and again until R1 becomes 0 (this means all numbers are added). Every time a number is added, then the R1 value is decremented.
- When R1 becomes 0, we come out of the loop and the result which is stored at R1 is copied into memory location SUM.

Condition Codes:

The processor keeps track of information about the results of various operations for use by subsequent conditional branch instructions. This is accomplished by recording the required information in individual bits, often called condition code flags. These flags are usually grouped together in a special processor register called the condition code register or status register. Individual condition code flags are set to 1 or cleared to 0, depending on the outcome of the operation performed.

Four commonly used flags are:

N (negative) Set to 1 if the result is negative; otherwise, cleared to 0

Z (zero) Set to 1 if the result is 0; otherwise, cleared to 0

V (overflow) Set to 1 if arithmetic overflow occurs; otherwise, cleared to 0

C (carry) Set to 1 if a carry-out results from the operation; otherwise, cleared to 0

The **N** and **Z** flags indicate whether the result of an arithmetic or logic operation is negative or zero. The N and Z flags may also be affected by instructions that transfer data, such as Move, Load, or Store.

The **V** flag indicates whether overflow has taken place.

The **C** flag is set to 1 if a carry occurs from the most significant bit position during an arithmetic operation. This flag makes it possible to perform arithmetic operations on operands that are longer than the word length of the processor.

The instruction Branch>0, tests one or more of the condition flags.



The IEEE Standard for Floating-Point Arithmetic:

The IEEE Standard for Floating-Point Arithmetic (IEEE 754) is a technical standard for floating-point computation which was established in 1985 by the Institute of Electrical and Electronics Engineers (IEEE). The standard addressed many problems found in the diverse floating point implementations that made them difficult to use reliably and reduced their portability. IEEE Standard 754 floating point is the most common representation today for real numbers on computers, including Intel-based PC's, Macs, and most UNIX platforms.

There are several ways to represent floating point number but IEEE 754 is the most efficient in most cases. IEEE 754 has 3 basic components:

1. The Sign of Mantissa –

This is as simple as the name. 0 represents a positive number while 1 represents a negative number.

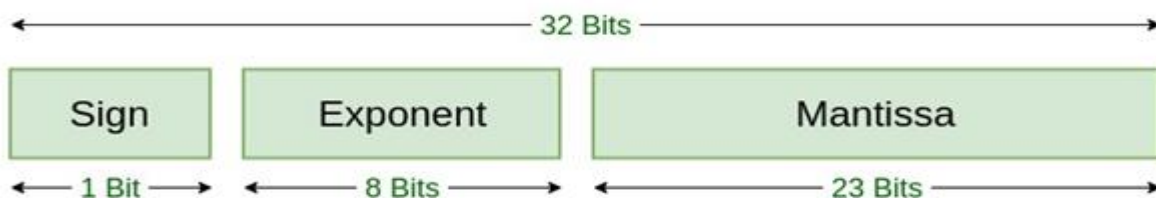
2. The Biased exponent –

The exponent field needs to represent both positive and negative exponents. A bias is added to the actual exponent in order to get the stored exponent.

3. The Normalized Mantissa –

The mantissa is part of a number in scientific notation or a floating-point number, consisting of its significant digits. Here we have only 2 digits, i.e. 0 and 1. So a normalized mantissa is one with only one 1 to the left of the decimal.

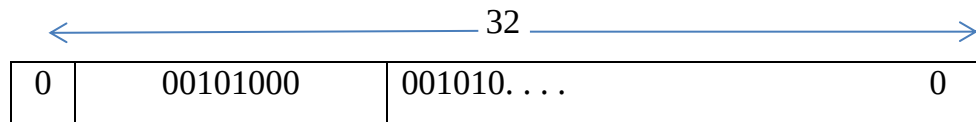
IEEE 754 numbers are divided into two based on the above three components: single precision and double precision.



Single Precision
IEEE 754 Floating-Point Standard



$$\text{Value represented} = (-1)^S \times 1.M \times (2)^{E-127}$$

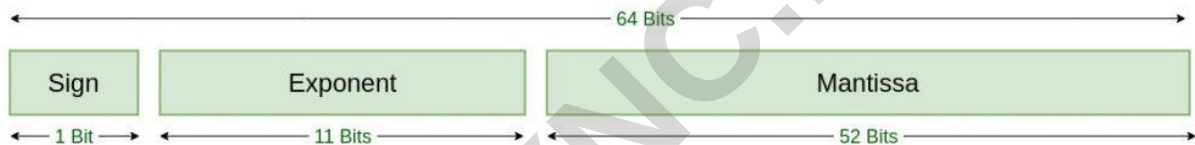
Example:

$$\text{Value} = (-1)^S \times 1.M \times (2)^{E-127}$$

$$\text{Value} = (-1)^0 \times 1.001010 \times (2)^{40-127}$$

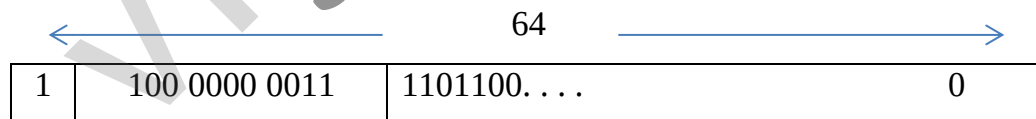
$$\text{Value} = (-1)^0 \times 1.001010 \times (2)^{-87}$$

$$\text{Value} = 1.001010 \times (2)^{-87}$$



Double Precision
IEEE 754 Floating-Point Standard

$$\text{Value represented} = (-1)^S \times 1.M \times (2)^{E-1023}$$

Example:

$$\text{Value} = (-1)^S \times 1.M \times (2)^{E-1023}$$

$$\text{Value} = (-1)^1 \times 1.1101100 \times (2)^{1027-1023}$$

$$\text{Value} = (-1)^1 \times 1.11001100 \times (2)^4$$

$$\text{Value} = -1.110011 \times (2)^4$$

Problem: For the following processor, obtain the performance.

Clock rate = 800 MHz

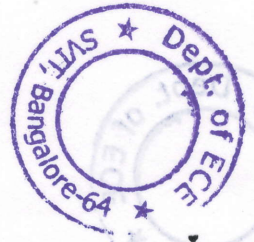
No. of instructions executed = 1000

Average no of steps needed / machine instruction = 20

Numbers :-

Numbers are represented in three systems. They are :-

- i) Sign and Magnitude
- ii) 1's Complement
- iii) 2's Complement.



Sign and Magnitude:

In sign and magnitude system, negative values are represented by changing the most significant bit (MSB) from 0 to 1.

Example: +5 is represented as 0101

-5 is represented as 1101

1's Complement:

In 1's complement representation, negative values are obtained by complementing each bit of the corresponding positive number.

Example: +5 is represented as 0101

-5 is represented as 1101

→ sign bit

Keep sign bit as it is, take 1's complement of remaining bits i.e.

1101 → -010

∴ 1010

∴ 1's complement of -5 is 1010.

2's Complement: 2's Complement of a number is done by adding 1 to the 1's Complement of that number.

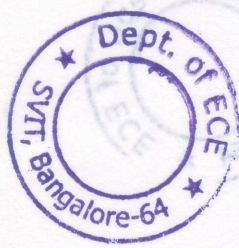
Example:

$$-0 \Rightarrow 1000$$

$$\begin{array}{r} 1000 \\ - 111 \leftarrow 1's \text{ Complement} \\ \hline \end{array}$$

$$\begin{array}{r} \\ + 1 \leftarrow \text{Adding 1 to obtain 2's} \\ \hline 1000 \end{array}$$

$$\boxed{-8}$$



Binary Signed Integer representation:-

b_3	b_2	b_1	b_0	Sign & Magnitude	1's Comp	2's Comp
0	1	1	1	+7	+7	+7
0	1	1	0	+6	+6	+6
0	1	0	1	+5	+5	+5
0	1	0	0	+4	+4	+4
0	0	1	1	+3	+3	+3
0	0	1	0	+2	+2	+2
0	0	0	1	+1	+1	+1
0	0	0	0	+0	+0	+0
1	0	0	0	-0	-7	-8
1	0	0	1	-1	-6	-7
1	0	1	0	-2	-5	-6
1	0	1	1	-3	-4	-5
1	1	0	0	-4	-3	-4
1	1	0	1	-5	-2	-3
1	1	1	0	-6	-1	-2
1	1	1	1	-7	-0	-1

Addition of numbers Using 2's Complement method

step 1: Take each decimal number into 2's complement

step 2: Add both numbers and if carry generate ignore.

a) +2 and +3

$$+2 \rightarrow 0010$$

$$+3 \rightarrow 0011$$

$$\begin{array}{r} 0010 \\ + 0011 \\ \hline 0101 \end{array} \quad \boxed{+5}$$

b) +4 $\rightarrow$ 0100

$$\begin{array}{r} 0100 \\ + 0110 \\ \hline 1010 \end{array}$$

c) +4 $\rightarrow$ 0100

-6 $\rightarrow$ 1110

$$\begin{array}{r} 2's \\ \text{Comp} \end{array} \rightarrow \begin{array}{r} -001 \\ +1 \\ \hline -010 \end{array}$$

$$-010 \rightarrow 1010$$

2's Comp of -6 $\rightarrow$

$$\begin{array}{r} 0100 \\ + 1010 \\ \hline 1110 \end{array}$$

$$\begin{array}{r} 1110 \end{array} \quad \boxed{-6} \text{ Answer.}$$

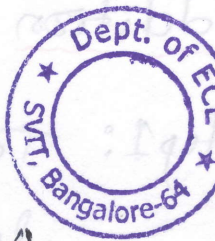
Sign bit $\rightarrow$

$$\begin{array}{r} +4 \\ -6 \\ \hline -2 \end{array}$$

-2 $\leftarrow$ Sign & Magnitude

-6 is 2's Comp $\rightarrow$

d) -5 and -2.



$$\begin{array}{rcl}
 -5 & \rightarrow & 1101 \\
 -2 & \rightarrow & 1010
 \end{array}
 \xrightarrow[2's \text{ comp}]{2's}
 \begin{array}{r}
 -010 \\
 +1 \\
 \hline
 -011 \rightarrow 1011
 \end{array}$$

$$\begin{array}{r}
 1101 \\
 +1 \\
 \hline
 1110
 \end{array}$$

$$\begin{array}{r}
 11011 \leftarrow 2's \text{ comp of } -5 \\
 + 1110 \leftarrow 2's \text{ comp of } -2 \\
 \hline
 11001
 \end{array}$$

ignore carry

1001

-1

Answer.

$$\begin{array}{r}
 -5 \\
 + \\
 -2 \\
 \hline
 -7 \xrightarrow[2's \text{ comp}]{2's} -1
 \end{array}$$

Subtraction of numbers using 2's Complement :-

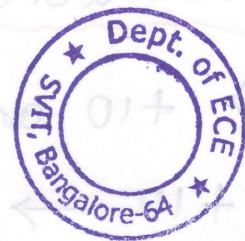
Step 1: Take each decimal number into 2's comp

Step 2: Subtract the numbers and if borrow occurs from MSB ignore.

a) -3 and -7

$$\begin{array}{rcl}
 -3 & \rightarrow & 1011 \\
 & & \xrightarrow[2's \text{ comp}]{2's} -100 \\
 & & +1 \\
 & & \hline
 & & -101 \xrightarrow[2's \text{ comp}]{2's} 1101
 \end{array}$$

$$\begin{array}{rcl}
 -7 & \rightarrow & 1111 \\
 & & \xrightarrow[2's \text{ comp}]{2's} -000 \\
 & & +1 \\
 & & \hline
 & & 1001 \xrightarrow[2's \text{ comp}]{2's} 1001
 \end{array}$$



$$\begin{array}{r}
 1101 \\
 - 1001 \\
 \hline
 0100 \quad \boxed{+4}
 \end{array}$$

Answer.

b) +2 and +4

$$+2 \rightarrow 0010$$

$$+4 \rightarrow 0100$$

$$\begin{array}{r}
 0010 \\
 + 0100 \\
 \hline
 1110
 \end{array}$$

$\boxed{-6}$

Answer is 1110

$$\begin{array}{r}
 +2 \\
 \oplus \\
 +4 \\
 \hline
 -2
 \end{array}$$

$$-2 \xrightarrow{2's \text{ comp}} \boxed{-6}$$

c) +6 and +3

$$+6 \rightarrow 0110$$

$$+3 \rightarrow 0011$$

$$\begin{array}{r}
 0110 \\
 + 0011 \\
 \hline
 0011
 \end{array}$$

$\boxed{+3}$

d) -7 and -5

$$-7 \rightarrow 1111$$

$$-5 \rightarrow 1101$$

$$\begin{array}{r}
 1111 \\
 \xrightarrow{2's \text{ comp}} \\
 -000
 \end{array}$$

$$\begin{array}{r}
 -000 \\
 +1 \\
 \hline
 -001
 \end{array}$$

$$-001 \rightarrow 1001$$

$$\begin{array}{r}
 1101 \\
 \xrightarrow{2's \text{ comp}} \\
 -010 \\
 +1 \\
 \hline
 -011
 \end{array}$$

$$-011 \rightarrow 1011$$

$$\begin{array}{r}
 1001 \\
 - 1011 \\
 \hline
 1110
 \end{array}$$

$$\begin{array}{r}
 1110 \\
 \boxed{-6}
 \end{array}$$

$\boxed{-6}$

Answer

Answer is 1110.

$$\begin{array}{r}
 -7 \\
 \oplus \\
 -5 \\
 \hline
 -2
 \end{array}$$

$$-2 \xrightarrow{2's \text{ comp}} -6$$

Subtraction of 5 bit number using 2's complement

a) +10 and -8

$$+10 \rightarrow 01010$$

$$-8 \rightarrow 11000 \xrightarrow[comp]{2's} \begin{array}{r} 0111 \\ +1 \\ \hline \end{array}$$

$$\begin{array}{r} 0111 \\ +1 \\ \hline 1000 \end{array} \rightarrow 11000$$

$$\begin{array}{r} 01010 \\ - 11000 \\ \hline \end{array}$$

$$\begin{array}{r} 110010 \\ \hline \end{array} \quad (+28)$$

ignore

$$\therefore 10010$$

Answer.

b) -15 and -9

$$-15 \rightarrow 11111 \xrightarrow[comp]{2's} \begin{array}{r} 0000 \\ +1 \\ \hline \end{array}$$

$$-9 \rightarrow 11001$$

$$\begin{array}{r} 0000 \\ +1 \\ \hline 10001 \end{array} \rightarrow 10001$$

$$\begin{array}{r} 11111 \\ \xrightarrow[comp]{2's} 0110 \\ +1 \\ \hline \end{array}$$

$$\begin{array}{r} 0110 \\ +1 \\ \hline 0111 \end{array} \rightarrow 10111$$

$$\begin{array}{r} 10001 \\ - 10111 \\ \hline \end{array}$$

$$\begin{array}{r} 111010 \\ \hline \end{array} \quad (-10)$$

ignore

$$\therefore 11010 \text{ Answer.}$$

c) -14 and +5

$$-14 \rightarrow 11110 \xrightarrow[comp]{2's} \begin{array}{r} 10001 \\ \hline \end{array}$$

$$+5 \rightarrow 00101$$

$$\begin{array}{r} 10001 \\ - 00101 \\ \hline \end{array}$$

$$01100$$

